

Hackathon Blueprint

Part 1: How to Prepare (The "Pre-Game" Setup)

You cannot figure out "how to connect to a database" during the hackathon. You need a **"War Kit"** ready before the timer starts.

1. Build Your "Golden Template":

- Create a skeleton project in your preferred stack (e.g., Spring Boot or MERN) *today*.
- **It must have:**
 - Database connection (Postgres/MongoDB) pre-configured.
 - Authentication boilerplate (JWT setup, Login/Signup APIs working).
 - Global Error Handling (so you don't write `try-catch` 50 times).
 - A `Dockerfile` ready to deploy.
- *Why?* This saves you the first 60 minutes of setup time.

2. Master the "Invisible" Tools:

- **Postman/Thunder Client:** Have collections ready to test your APIs instantly.
- **Ngrok:** If you need to expose your local server to a webhook (common in payment/chat problems).
- **Diagramming Tool:** Learn **Excalidraw**. You will likely need to submit a system architecture diagram.

Part 2: How to Tackle a Problem Statement (The Framework)

When the problem statement drops (usually a PDF), do not touch your keyboard for the first 30 minutes. Use the **"Nouns & Verbs"** method.

Step 1: Deconstruct (Nouns & Verbs)

Read the problem. Circle the **Nouns** (Entities) and **Verbs** (Actions).

- *Example Problem:* "Design a Splitwise clone where users can add expenses and split bills."
- **Nouns:** User, Expense, Group, Settlement.
- **Verbs:** Add (Expense), Split (Bill), Settle (Debt).

Result: You immediately have your **Database Schema** (The Nouns become Tables) and your **API Endpoints** (The Verbs become Controllers).

Step 2: The "MVP" Filter (Crucial)

Hackathons have tight deadlines. You must distinguish between **"Core"** and **"Fancy"**.

- *Core:* Adding an expense, calculating who owes whom. (DO THIS).
- *Fancy:* Email notifications, currency conversion, login with Google. (IGNORE THIS initially).

Step 3: Schema First, Code Later

Never start coding API logic without a DB schema.

- Draw the relationships: *User 1:N Expense, Group 1:N User.*
- If you get the Schema wrong, you will rewrite code 3 hours later.

Part 3: Architecture & Tech Stack Strategy

Don't over-engineer. Judges want **Clean**, not **Complex**.

1. The "Modular Monolith" (Recommended)

Do not build Microservices for a 24-hour hackathon. It is a trap.

- **Structure:**
 - `src/controllers` (API entry points)
 - `src/services` (Business logic - *Put all complex math here*)
 - `src/models` (Database schema)
 - `src/utils` (Helper functions)

2. Tech Stack Selection

- **Rule of Thumb:** Use what you can debug at 3 AM.
- **Backend:** Java Spring Boot or Node.js (Express/NestJS). *Avoid Python/Django unless it's a Data Science/AI problem.*
- **Database:** PostgreSQL (Safe choice) or MongoDB (Only if the data structure is highly unpredictable).
- **Frontend:** React + Tailwind CSS (Speed is key).

3. The "X-Factor" (What Judges Look For)

If two candidates finish the task, the one with these three things wins:

1. **Handling Concurrency:** What happens if two users click "Pay" at the exact same millisecond? (Mention Database Locking or synchronized blocks).
2. **Input Validation:** Don't let a user pay **-100** rupees.
3. **The README:** Your code might be great, but if your README doesn't explain how to run it, you lose. Include:
 - *Prerequisites* (Docker, Node version).
 - *Steps to run.*
 - *API Documentation* (Screenshots of Postman or Swagger link).

Example: Tackling a "Parking Lot" Problem

Problem: Design a system to assign spots to cars entering a mall.

1. **Architecture:**
 - **Service Layer:** `ParkingService` (Logic to find the nearest empty spot).
 - **Data Layer:** `SpotRepository` (SQL query to find `WHERE is_occupied = false`).
2. **Concurrency:**

- *Challenge:* Two cars enter Gate A and Gate B simultaneously. Both are assigned Spot #5.
- *Solution:* Use Database Row Locking (`SELECT ... FOR UPDATE` in SQL) to lock Spot #5 while assigning it.

3. **Code Quality:**

- Don't use "Magic Numbers". Instead of `if (type == 1)`, use Enums: `if (type == VehicleType.SUV)`.

Summary Checklist for the Day Of:

- ☐ **Hour 0-1:** Read, Diagram, Schema Design.
- ☐ **Hour 1-4:** Build the "Happy Path" (Core features work).
- ☐ **Hour 5:** Handle Edge Cases (Negative numbers, Null inputs).
- ☐ **Hour 6:** Polish (Comments, README, cleanup).